OpenWrite

Open-Source AI Writing Copilot

Software Requirements Specification and Technical Design Document

Version 1.0 – MVP Baseline

31 October 2026

Project category: Generative AI / Natural Language Processing / Browser Extension

Status: Proposed design; implementation not yet started

Contents

1 Executive Summary **1**
1.1 Document control . . . 1

2 Problem, Vision and Goals **2**
2.1 Problem statement . . . 2
2.2 Product vision . . . 2
2.3 Objectives . . . 2
2.4 Success criteria for MVP . . . 2

3 Scope and Assumptions **3**
3.1 In scope: MVP . . . 3
3.2 Out of scope: MVP . . . 3
3.3 Assumptions and dependencies . . . 3

4 Stakeholders, Personas and Use Cases **4**
4.1 Stakeholders . . . 4
4.2 Personas . . . 4
4.3 Primary use cases . . . 4

5 Functional Requirements **5**
5.1 Editor detection and interaction . . . 5
5.2 Autocomplete . . . 5
5.3 AI, settings, and operational behavior . . . 6

6 Non-Functional Requirements **7**

7 Technology Stack and Rationale **8**
7.1 Why Express instead of FastAPI? . . . 8
7.2 Provider selection criteria . . . 8

8 System Architecture **9**
8.1 Logical component diagram . . . 9
8.2 Component responsibilities . . . 9
8.3 Data flow: rewrite . . . 10
8.4 Data flow: autocomplete . . . 10

9 API Contract **11**
9.1 Endpoints . . . 11
9.2 Request example . . . 11
9.3 Success response . . . 11
9.4 Error response . . . 11
9.5 Validation and provider behavior . . . 12

10 Interface and Interaction Design **13**
10.1 Main UI elements . . . 13
10.2 Editor compatibility strategy . . . 13
10.3 Accessibility . . . 13

11 Security, Privacy and Responsible AI **14**
11.1 Threats and controls . . . 14
11.2 Privacy principles . . . 14
11.3 Responsible AI . . . 14

12 Testing and Acceptance **15**
12.1 Test layers . . . 15
12.2 Acceptance test cases . . . 15
12.3 Performance measurement . . . 16

13 Development Plan and Milestones **17**
13.1 Definition of done . . . 17

14 Repository Structure **18**

15 Risks, Trade-offs and Future Roadmap **19**
15.1 Risk register . . . 19
15.2 Post-MVP enhancements . . . 19

16 Open-Source and Release Strategy **20**
16.1 Licensing . . . 20
16.2 Contributor onboarding . . . 20
16.3 Release checklist . . . 20

17 References and Further Reading **21**

A Glossary **22**

B Initial Configuration Example **23**

1 Executive Summary

OpenWrite is a proposed open-source browser extension that helps users write faster and more clearly inside supported web editors. Its two signature interactions are (i) context-aware text continuation and (ii) a selection-triggered pen toolbar for grammar correction, paraphrasing, and tone adjustment. The first release uses existing third-party language-model APIs; it does *not* train, fine-tune, or host a custom model. The system consists of a React/TypeScript extension, a small Node.js/Express API, and a configurable AI provider adapter. User consent, predictable text editing, and minimal permissions are foundational requirements.

1.1 Document control

Project name	OpenWrite (working name; naming and trademark availability to be checked)
Document type	Software Requirements Specification (SRS) and High-Level Technical Design
Version	1.0
Release target	Browser-extension MVP / private beta
Audience	Project developer, contributors, testers, maintainers
Status	Proposed; performance figures are initial targets, not verified results

2 Problem, Vision and Goals

2.1 Problem statement

Writing across websites often requires switching to separate AI chat interfaces for sentence completion, grammar correction, and tone adjustments. Existing editor integrations vary considerably; users need a low-friction tool that offers suggestions directly where they type while preserving their control over edits and data.

2.2 Product vision

A transparent, accessible, open-source writing copilot that offers useful, opt-in suggestions in common browser text fields, with provider flexibility and a roadmap toward privacy-preserving inference.

2.3 Objectives

1. Provide selection-based grammar, rewrite, and tone transformations with preview and explicit acceptance.
2. Offer short context-aware completions without blocking typing.
3. Build on the developer's existing React, TypeScript, Node.js, and Express skills.
4. Avoid database and user accounts in the initial private beta.
5. Make model providers replaceable through a single backend abstraction.
6. Document security, privacy, limitations, and editor compatibility honestly.

2.4 Success criteria for MVP

- A user can select text in a supported editor, open the pen toolbar, request a correction, preview it, and replace exactly the intended selection.
- A user can receive and explicitly accept a short autocomplete suggestion in a supported editor.
- The extension can be disabled globally or for a website; no model request occurs when disabled.
- Requests to the model use a server-held credential, not a credential embedded in the extension.
- Basic automated tests and a documented manual compatibility matrix are available.

3 Scope and Assumptions

3.1 In scope: MVP

- Chromium-based browsers supported by WXT's Manifest V3 output, initially Chrome.
- Plain `textarea`, eligible text `input`, and basic `contenteditable` editors.
- Grammar correction, paraphrasing, professional/friendly/concise tone rewriting.
- Autocomplete via a small suggestion popup; inline ghost text is a later enhancement.
- Explicit accept/reject, undo-compatible insertion where feasible, keyboard controls.
- Popup settings, local extension storage, and website exclusions.
- Node.js/Express API with provider adapter and basic operational protections.

3.2 Out of scope: MVP

- Training or fine-tuning any AI model; proprietary training datasets; GPU provisioning.
- A dedicated mobile keyboard, desktop-wide editor, or native mobile apps.
- Guaranteed compatibility with Google Docs, canvas editors, all iframes, or every rich-text framework.
- Persistent server-side text history, accounts, collaborative editing, or billing.
- Automated fact checking, plagiarism detection, or guaranteed factual correctness.
- Always-on production-grade free inference; provider charges and quotas may apply.

3.3 Assumptions and dependencies

Users have a supported browser and internet connectivity for cloud inference. A provider offers a suitable text-generation API, subject to pricing, availability, content policy, and model-license constraints. Backend hosting is required for external beta testers; Colab and Kaggle are not production API hosting solutions. Model outputs are untrusted suggestions and may be incorrect.

4 Stakeholders, Personas and Use Cases

4.1 Stakeholders

Developer/maintainer; end users (students, professionals, job applicants); open-source contributors; AI inference provider; browser extension distribution platform.

4.2 Personas

Student Wants clearer assignment messages and email drafts without switching tabs.

Job seeker Wants concise, professional application messages and follow-ups.

Professional Wants quick completions and respectful tone changes in everyday writing.

Contributor Wants understandable architecture, local setup, and tests.

4.3 Primary use cases

ID	Use case	Expected behavior
UC-01	Fix selected text	Select text, click pen, choose grammar, review result, accept or dismiss.
UC-02	Rephrase	Choose rewrite mode; preserve core meaning and named entities.
UC-03	Change tone	Select professional, friendly, or concise style and preview result.
UC-04	Autocomplete	Pause typing; see suggested continuation; Tab accepts, Esc dismisses when suggestion has focus/ownership.
UC-05	Preferences	Disable all suggestions or exclude a website; change completion length.
UC-06	Failure handling	Timeout or provider failure shows nonblocking error and never modifies text.
UC-07	Privacy	Disable transmission on excluded/sensitive fields and show whether cloud inference is enabled.

5 Functional Requirements

Requirements use **MUST** for MVP obligations and **SHOULD** for desirable behavior. IDs support traceability.

5.1 Editor detection and interaction

ID	Priority	Requirement
FR-01	MUST	Detect supported editable fields on enabled websites without intercepting unrelated page content.
FR-02	MUST	Read selected text and retain sufficient editor/selection identity to verify a replacement remains valid.
FR-03	MUST	Show a nonintrusive pen control near a supported text selection.
FR-04	MUST	Provide grammar, rephrase, and tone actions in the pen toolbar.
FR-05	MUST	Preview the model output before replacing selected text.
FR-06	MUST	Replace only the intended selected range; reject application if editor or selection has changed incompatibly.
FR-07	MUST	Never silently rewrite, submit, or send user text.
FR-08	SHOULD	Preserve focus, caret position, and editor undo behavior as supported by the target editor.
FR-09	MUST	Avoid password, OTP, payment-card, and explicitly excluded fields.
FR-10	SHOULD	Handle dynamically inserted editors and remove UI listeners on teardown.

5.2 Autocomplete

ID	Priority	Requirement
FR-11	MUST	Request completion only after a configurable typing pause; default initial debounce 350 ms.
FR-12	MUST	Use bounded context preceding the cursor; do not collect the entire page.
FR-13	MUST	Show completion in a popup without altering the document until accepted.
FR-14	MUST	Provide explicit acceptance and dismissal; keyboard shortcuts must not override existing editor behavior unexpectedly.
FR-15	MUST	Discard stale responses if input, caret, focus, or editor version changes.
FR-16	MUST	Limit completion length by a selected short/medium mode.

ID	Priority	Requirement
FR-17	SHOULD	Suppress low-quality, empty, repetitive, or prefix-duplicating suggestions.
FR-18	SHOULD	Permit users to disable autocomplete independently of rewriting.

5.3 AI, settings, and operational behavior

ID	Priority	Requirement
FR-19	MUST	Backend supports separate task types: `autocomplete`, `grammar`, `rewrite`, `tone`.
FR-20	MUST	Validate task, text type, size, and supported options before provider calls.
FR-21	MUST	Keep provider API keys server-side and out of the extension bundle.
FR-22	MUST	Return normalized success/error responses independent of provider.
FR-23	MUST	Provide extension-wide enable/disable and per-site exclusions in local storage.
FR-24	MUST	Obtain meaningful opt-in consent before sending editor text to a remote model service.
FR-25	MUST	Display actionable error states for offline, timeout, quota, and provider failures.
FR-26	SHOULD	Allow provider/model configuration via backend environment variables.
FR-27	SHOULD	Provide a settings view describing cloud processing and privacy practices.
FR-28	SHOULD	Record anonymous aggregate performance metrics only if separately enabled; no raw text logs by default.

6 Non-Functional Requirements

ID	Attribute	Initial acceptance target
NFR-01	Responsiveness	Extension UI responds to local interactions within 100 ms in ordinary test conditions; AI latency is displayed separately.
NFR-02	AI latency	Aim for p95 autocomplete suggestion within 3 s and rewrite within 8 s on a selected provider; measure rather than guarantee.
NFR-03	Reliability	Failed or late requests never overwrite current user content; safe fallback is no change.
NFR-04	Privacy	No raw editor text persisted on backend by default; send only task-relevant bounded text after consent.
NFR-05	Security	TLS for deployed API; credentials in server secrets; least-privilege extension permissions; input validation and rate limits.
NFR-06	Accessibility	Keyboard operability, visible focus, readable contrast, screen-reader labels, and no focus traps.
NFR-07	Compatibility	Publish a tested matrix for Chrome and supported field types; unsupported editors fail gracefully.
NFR-08	Maintainability	TypeScript types, modular adapters, linting, documented environment variables, and automated tests.
NFR-09	Scalability	Backend is stateless where practical; quota and concurrency controls protect provider budget.
NFR-10	Observability	Structured error logs with request IDs, durations and status; redact text, tokens, and personal content.
NFR-11	Portability	Provider adapter isolates vendor-specific request/response formats.
NFR-12	Cost	Configurable request and token limits; usage alerts before public release.

7 Technology Stack and Rationale

Layer	Technology	Responsibility
Extension framework	WXT + Manifest V3	Build tooling, entrypoints, browser extension packaging.
UI	React + TypeScript	Popup, settings, pen toolbar, suggestion box.
Styles	Tailwind CSS	Consistent, lightweight UI.
Browser APIs	DOM, Selection/Range, chrome.storage, messaging	Text detection, editing, preferences, and inter-process communication.
Backend runtime	Node.js	Run JavaScript API service.
Backend framework	Express.js	Request validation, task routes, error handling, provider dispatch.
AI inference	Existing hosted text-generation API	Generate corrections, rewrites, and completions; no model training.
Provider adapter	JavaScript module	Map normalized tasks to provider API.
Persistence	chrome.storage.local	Store client preferences only; no server database for MVP.
Testing	Vitest, Playwright, Supertest	Unit, editor integration, and API tests.
Source control	Git + GitHub	Open-source collaboration, issues, pull requests, CI.
Hosting	Node-compatible hosting provider	Host Express API for testers; pricing and limits to be verified.

7.1 Why Express instead of FastAPI?

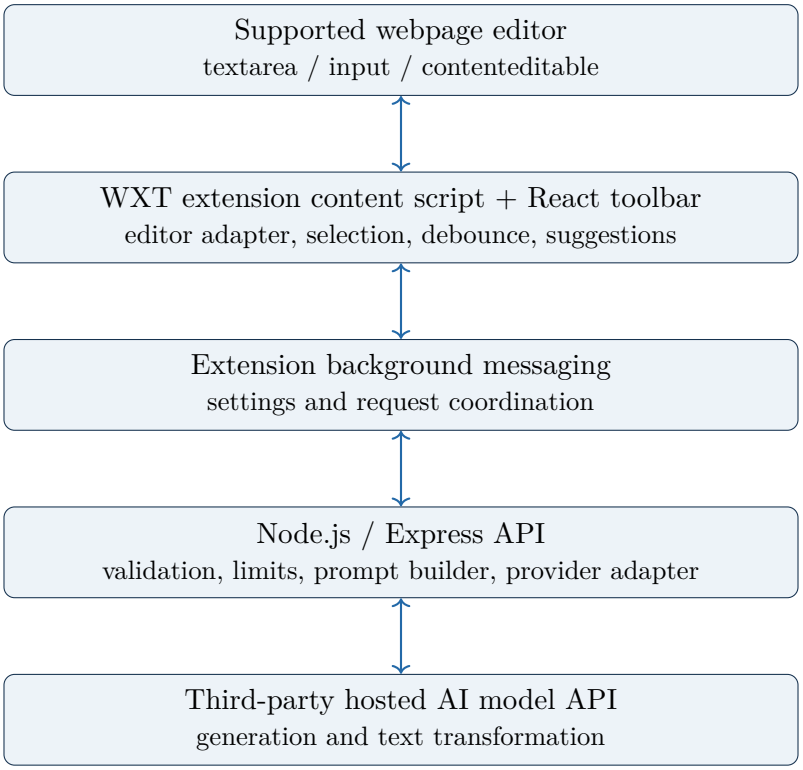
Express keeps the product entirely in the JavaScript/TypeScript ecosystem. It is familiar, lightweight, and sufficient because OpenWrite delegates inference to an external model provider. Python is not required unless the project later introduces model training or local Python inference services.

7.2 Provider selection criteria

Compare providers on quality for correction and continuation, measured p50/p95 latency, cost per request/token, API reliability, regional availability, data retention terms, rate limits, moderation behavior, and model/license terms. Do not hard-code the application to a single vendor.

8 System Architecture

8.1 Logical component diagram



8.2 Component responsibilities

Content script Detect supported editors; manage selection/caret, UI placement, and insertion. Never hold provider secrets.

Editor adapter Standardize `getText`, `getSelection`, `replaceSelection`, and `getCaretRect`. Start with native fields and simple contenteditable.

Background Coordinate messages and requests as needed; do not assume a Manifest V3 worker stays alive indefinitely.

Express API Validate, authenticate/rate-limit as appropriate, build task prompts, call provider, normalize output, handle failures.

AI provider Produce untrusted text proposals; not a source of guaranteed truth.

8.3 Data flow: rewrite

1. User highlights text and clicks pen action.
2. Extension verifies supported field, allowed site, and consent status.
3. Extension captures selected text and a version/selection marker.
4. Background/API request contains task, bounded text, and mode.
5. Express validates and forwards a prompt to the configured provider.
6. Provider returns candidate text; backend normalizes response.
7. Extension displays preview; user accepts or dismisses.
8. Before replacement, extension revalidates editor identity and selection. If invalid, request reselection.

8.4 Data flow: autocomplete

1. User types; content script waits for a debounce interval.
2. If field/site is eligible and consented, capture limited preceding context.
3. Attach monotonic request sequence number and editor revision.
4. Call backend; cancel or ignore outdated responses.
5. If response still matches active editor state, display suggestion popup.
6. On explicit acceptance, insert text using the editor adapter; otherwise leave content unchanged.

9 API Contract

9.1 Endpoints

Path	Method	Purpose
`/api/health`	GET	Liveness check; does not expose secrets.
`/api/autocomplete`	POST	Generate bounded continuation.
`/api/grammar`	POST	Correct selected text.
`/api/rewrite`	POST	Paraphrase selected text.
`/api/tone`	POST	Rewrite to requested tone.

9.2 Request example

```
POST /api/rewrite
Content-Type: application/json
{
  "text": "i wanted to ask about internship",
  "mode": "professional",
  "requestId": "client-generated-id"
}
```

9.3 Success response

```
{
  "requestId": "client-generated-id",
  "result": "I would like to inquire about the internship.",
  "task": "rewrite"
}
```

9.4 Error response

```
{
  "requestId": "client-generated-id",
  "error": {
    "code": "PROVIDER_TIMEOUT",
    "message": "Suggestion unavailable. Please retry."
  }
}
```

9.5 Validation and provider behavior

Require nonempty string input, permitted mode, maximum input size (initial limit: 4,000 characters for rewrite and smaller bounded autocomplete context), and strict server-side request timeout. Use a model-specific prompt that says return only the corrected text or continuation; still validate and clean outputs. Do not rely on prompts as a security boundary. Public deployment requires authentication/abuse protection, per-client quotas, and cost monitoring. Extension origins and CORS must be configured for the actual deployed client; avoid wildcard access to a paid unauthenticated endpoint.

10 Interface and Interaction Design

10.1 Main UI elements

- **Toolbar:** small pen icon near selection; actions Grammar, Rephrase, Professional, Friendly, Concise.
- **Preview card:** original vs suggested text, Apply, Dismiss, Retry; show loading and errors.
- **Autocomplete card:** compact suggestion below the editor; Tab/Click to accept, Esc to dismiss where safe.
- **Extension popup:** master toggle, autocomplete toggle, completion length, per-site exclusion, privacy link.
- **First-run disclosure:** explains which text may be sent to a remote model and requires opt-in before first request.

10.2 Editor compatibility strategy

Start with native `textarea` and plain text `input`, then simple `contenteditable`. Rich editors may maintain their own document state, so direct DOM replacement can break undo/history. Use explicit adapters and test each integration. Shadow roots, cross-origin frames, virtual editors, and canvas-based editors may be unsupported in v0.1.

10.3 Accessibility

All controls must have descriptive accessible labels, visible focus indication, keyboard navigation, appropriate contrast, and readable error messages. Avoid hijacking Tab when no suggestion is actively available. Respect reduced-motion preferences.

11 Security, Privacy and Responsible AI

11.1 Threats and controls

Risk	Mitigation
Leaked API token	Store provider keys only in server environment/secrets; never ship to extension.
Sensitive text exposure	Consent before remote processing, bounded context, sensitive-field exclusions, per-site controls.
Malicious page content	Treat DOM text as untrusted; render suggestions as text, not injected HTML.
Prompt injection	Treat page text as data, not instructions; limit task scope and validate output.
Uncontrolled API cost	Auth or limited beta access, rate limits, quotas, request caps, monitoring.
Stale response overwrites	Editor revision and request sequence checks before display and apply.
Data retention	No raw text logging by default; document provider processing/retention separately.
Excessive permissions	Request only necessary host and extension permissions; consider optional site permissions.
Supply-chain risk	Dependency updates, lockfile, automated audit, review external packages.

11.2 Privacy principles

Minimize collection, transmit only when a feature is used and user has opted in, explain the selected provider, permit global and site-specific disabling, and avoid storing raw text. Any future history/sync feature requires a separate design and explicit opt-in. The public privacy notice must match actual implementation and provider terms.

11.3 Responsible AI

Show AI output as a suggestion, not authoritative truth. Preserve original text until acceptance; warn that grammar and rewriting may change meaning. Avoid claims of perfect grammar, privacy, or factual accuracy. Provide a quick way to dismiss and report bad suggestions.

12 Testing and Acceptance

12.1 Test layers

- **Unit:** prompt builder, validators, request state, debounce, output normalization, editor adapter utilities.
- **API integration:** mocked provider success, invalid input, timeout, quota, malformed response, authentication failure.
- **Browser integration:** native textarea, text input, simple contenteditable, selection loss, rapid typing, dynamic editor insertion.
- **Manual:** accessibility, undo, copy/paste, keyboard navigation, slow network, site exclusions, first-run consent.
- **Security:** secrets absent from extension bundle, no text in logs, disallowed fields ignored, rate limits enforced.

12.2 Acceptance test cases

ID	Scenario	Expected result
AT-01	Select text in textarea and request grammar	Preview appears; Apply changes only selected text.
AT-02	Move caret before rewrite response arrives	Stale result cannot silently replace another selection.
AT-03	Type rapidly while autocomplete requests overlap	Only latest eligible suggestion appears.
AT-04	Press Esc with suggestion visible	Suggestion disappears; editor text unchanged.
AT-05	Disable extension on a website	No AI requests or writing controls on excluded site.
AT-06	Select text in password field	No toolbar, extraction, or provider call.
AT-07	Provider returns timeout	Error shown; existing text preserved.
AT-08	Provider token missing on backend	API fails safely without exposing secret details.
AT-09	Request text exceeds configured limit	API returns validation error; provider not called.
AT-10	First use without cloud consent	No remote transmission until explicit opt-in.

12.3 Performance measurement

Track local UI interaction latency, model round-trip latency (p50/p95), request failure rate, completion acceptance rate, and user-reported edit correctness. Do not collect actual writing content for analytics without explicit additional consent.

13 Development Plan and Milestones

Phase	Activities	Deliverable / exit criterion
1: Foundation	Initialize WXT/React/TypeScript; implement extension popup and simple editor detection.	Toolbar works on native textarea with mock output.
2: Rewrite MVP	Build Express API and provider adapter; add grammar/rephrase/tone and preview/accept.	End-to-end selection-to-replacement flow passes tests.
3: Autocomplete	Debounce, bounded context, stale request handling, suggestion popup, accept/dismiss.	Functional completion in supported editors.
4: Hardening	Site controls, consent, validation, timeouts, rate limits, accessibility, editor tests.	Private beta readiness checklist passes.
5: Open source	README, contribution guide, license review, CI, issue templates, release notes.	Installable GitHub beta with known limitations.

13.1 Definition of done

A feature is done only when behavior is implemented, user-visible error states exist, tests cover expected and failure paths, security/privacy impact is reviewed, and documentation is updated. Publishing to the Chrome Web Store is optional for the first beta and requires compliance with store policies.

14 Repository Structure

```
openwrite/
  extension/
    entrypoints/
      background.ts
      content.ts
      popup/
    components/
      PenToolbar.tsx
      SuggestionCard.tsx
    adapters/
      textarea.ts
      contenteditable.ts
    lib/
      api.ts
      settings.ts
  server/
    src/
      index.ts
      routes/
      providers/
      prompts/
      middleware/
    .env.example
  docs/
    architecture.md
    privacy.md
    compatibility.md
  tests/
  .github/workflows/
  README.md
  CONTRIBUTING.md
  LICENSE
```

15 Risks, Trade-offs and Future Roadmap

15.1 Risk register

Risk	Severity	Response
Provider latency and cost	High	Debounce, quotas, model comparisons, short outputs, graceful failures.
Editor compatibility	High	Support matrix, adapter pattern, conservative MVP scope.
Privacy concerns	High	Explicit consent, exclusions, bounded text, clear policy.
Incorrect model suggestions	Medium	Preview, manual acceptance, meaning-preservation tests.
Provider changes/outages	Medium	Provider abstraction, retry policy, configuration.
Scope creep	Medium	Freeze v0.1 feature set; track later features separately.

15.2 Post-MVP enhancements

- Inline ghost text at caret, richer keyboard controls, multiple candidate suggestions.
- Better Gmail, LinkedIn, and WhatsApp Web integrations after targeted testing.
- Roman Urdu / English rewriting and multilingual support.
- Personal dictionary and configurable writing profiles stored locally.
- Optional local inference and user-supplied provider credentials with careful key handling.
- Quick replies, summarization, explanations of grammar changes.
- Optional opt-in feedback and privacy-preserving evaluation metrics.
- Model fine-tuning as a *separate future research project*, not part of the current architecture.

16 Open-Source and Release Strategy

16.1 Licensing

Select a code license (e.g., MIT or Apache-2.0) after reviewing project goals. This license covers original source code, not automatically the third-party model weights, SDKs, icons, datasets, or service terms. Document external dependencies and attribution requirements.

16.2 Contributor onboarding

Provide a README with feature list, screenshots, prerequisites, setup, environment variable examples, test commands, supported editors, security disclosure instructions, and roadmap. Add a CONTRIBUTING guide, issue templates, and a code of conduct when collaborators join.

16.3 Release checklist

- Secrets removed from tracked files and built extension assets.
- Production API access protected against abuse; request limits and cost controls enabled.
- First-run consent and privacy notice implemented and reviewed.
- Chrome permissions justified; only required sites enabled.
- Browser compatibility matrix and known limitations published.
- CI checks, unit/integration tests, manual smoke tests pass.
- Versioned build artifacts, release notes, and rollback plan available.

17 References and Further Reading

1. Chrome Extensions documentation: https://developer.chrome.com/docs/extensions/
2. WXT documentation: https://wxt.dev/
3. Express documentation: https://expressjs.com/
4. Hugging Face Inference Providers: https://huggingface.co/docs/inference-providers/
5. MDN Selection API: https://developer.mozilla.org/en-US/docs/Web/API/Selection
6. MDN Range API: https://developer.mozilla.org/en-US/docs/Web/API/Range
7. Playwright: https://playwright.dev/

A Glossary

GenAI Generative artificial intelligence; produces new content based on prompts.

NLP Natural language processing; computational analysis and generation of language.

LLM Large language model; pretrained model used for text generation and transformation.

Content script Extension code that executes in the context of permitted web pages.

Manifest V3 Current Chrome extension architecture and permission model.

Debounce Delaying an action until a short quiet period after repeated events.

Editor adapter Interface that hides differences among text-editing implementations.

Inference Running a trained model to generate an output.

MVP Minimum viable product; smallest coherent usable release.

SRS Software Requirements Specification.

B Initial Configuration Example

```
# server/.env.example
PORT=3001
AI_PROVIDER=huggingface
AI_MODEL=provider-supported-model-id
AI_API_KEY=replace-with-secret
REQUEST_TIMEOUT_MS=12000
MAX_REQUESTS_PER_MINUTE=20
```

These values are illustrative. Verify actual provider endpoint, supported model, credential naming, pricing, and production limits before deployment.